

Article

An Improved Soft Actor–Critic Task Offloading and Edge Computing Resource Allocation Algorithm for Image Segmentation Tasks in the Internet of Vehicles

Wei Zou ¹, Haitao Yu ², Boran Yang ^{1,*}, Aohui Ren ¹ and Wei Liu ¹¹ School of Artificial Intelligence, Chongqing University of Technology, Chongqing 401135, China; jakeyv067@gmail.com (W.Z.); renaohui_cqut@163.com (A.R.); veileno@cqut.edu.cn (W.L.)² China Satellite Network Exploration Co., Ltd., Chongqing 401121, China; ht_yu@outlook.com

* Correspondence: yangbr@cqut.edu.cn

Abstract

This paper addresses the challenge of offloading resource-intensive image segmentation tasks and allocating computing resources within the Internet of Vehicles (IoV) using edge-based AI. To overcome the limitations of onboard computing in smart vehicles, this study develops an efficient edge computing resource allocation system. The core of this system is an improved model-free soft actor–critic (iSAC) algorithm, which is enhanced by incorporating prioritized experience replay (PER). This PER-iSAC algorithm is designed to accelerate the learning process, maintain stability, and improve the efficiency and accuracy of computation offloading. Furthermore, an integrated computing and networking scheduling framework is employed to minimize overall task completion time. Simulation experiments were conducted to compare the PER-iSAC algorithm against baseline algorithms (Standard SAC and PPO). The results demonstrate that the proposed PER-iSAC significantly reduces task allocation error rates and optimizes task completion times. This research offers a practical engineering solution for enhancing the computational capabilities of IoV systems, thereby contributing to the development of more responsive and reliable autonomous driving applications.

Keywords: edge computing; image segmentation; task offloading; computation resource allocation; deep reinforcement learning; soft actor–critic; prioritized experience replay



Academic Editor: Grzegorz Sierpiński

Received: 15 April 2025

Revised: 8 June 2025

Accepted: 23 June 2025

Published: 25 June 2025

Citation: Zou, W.; Yu, H.; Yang, B.; Ren, A.; Liu, W. An Improved Soft Actor–Critic Task Offloading and Edge Computing Resource Allocation Algorithm for Image Segmentation Tasks in the Internet of Vehicles. *World Electr. Veh. J.* **2025**, *16*, 353. <https://doi.org/10.3390/wevj16070353>

Copyright: © 2025 by the authors. Published by MDPI on behalf of the World Electric Vehicle Association. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Under the synergistic evolution of 5G wireless communication and artificial intelligence (AI), transportation systems are undergoing a profound transformation toward distributed intelligence and cooperative autonomy. The integration of vehicle communication technologies, edge computing, and AI marks a paradigm shift in the architecture and capabilities of intelligent transportation systems (ITS) [1]. Unlike traditional ITS models, which are limited by siloed sensing and centralized cloud-based decision making, the new generation of connected vehicles operates within a dynamic, low-latency, and resource-optimized network environment enabled by edge computing [2,3]. Edge computing decentralizes computational resources by moving core cloud functionalities closer to data sources, such as roadside units (RSUs) [4], which enables the rapid processing of data from local vehicles and infrastructure.

The promise of edge computing lies not only in offloading computationally intensive tasks—such as image segmentation, sensor fusion, and path planning—from onboard

units to proximate edge nodes, but also in enabling real-time coordination across the vehicle-to-everything (V2X) spectrum. This coordination spans vehicle-to-vehicle (V2V), vehicle-to-infrastructure (V2I), vehicle-to-network (V2N), and vehicle-to-pedestrian (V2P) communications, laying the foundation for collective behavior, system-level optimization, and safety-critical interactions [5,6]. By aggregating high-frequency data streams from multiple sources, including cameras, lidars, GPS, and vehicle dynamics [7], edge servers construct a spatiotemporally rich digital twin of local traffic environments. This enables predictive analytics and collaborative decision making across agents, vastly improving situational awareness and operational efficiency [8].

However, this distributed intelligence paradigm introduces significant challenges. First, the surge in connected vehicles and their sensing capabilities results in explosive data generation—modern autonomous vehicles can produce terabytes of data every day [9]. Processing this volume in real time with stringent latency and energy constraints requires efficient task offloading, computing, and bandwidth allocation mechanisms [3]. Onboard computing power, though improving, cannot keep pace with the rising demand for high-precision applications such as semantic image segmentation, multi-object tracking, and real-time inference [10], leading to bottlenecks in performance and cost escalation.

Edge computing, therefore, emerges as a critical architectural solution. By decentralizing computational resources and bringing them closer to data sources, it reduces transmission latency, alleviates cloud dependence, and supports the responsiveness required for safety and efficiency in vehicular environments. Yet the success of such a system hinges on intelligent resource scheduling and task management across heterogeneous edge nodes and dynamic network conditions.

Through this design, we shift the edge server's role from a passive computational assistant to an active orchestrator of cooperative transport behavior. This holistic framework not only accelerates perception and decision-making tasks but also facilitates system-level coordination for platooning, intersection management, and hazard mitigation. The contributions of this research thus extend beyond algorithmic efficiency to propose a foundational infrastructure model for next-generation intelligent transportation systems.

To address this issue, this paper analyzed mainstream task allocation and resource scheduling algorithms and constructed a more efficient edge computing resource allocation system based on the model-free deep reinforcement learning (DRL) algorithm with maximum entropy, i.e., soft actor–critic (SAC) [11]. The improved soft actor–critic (iSAC) algorithm proposed optimizes the allocation of computing resources based on the principles of time priority and resource priority within the CPN. Through an integrated computing and bandwidth scheduling framework, iSAC achieves unified scheduling of the computing network, enhancing the rational allocation of resources. The main contributions of this paper are as follows:

- (1) Computing resource allocation for connected cars involves decisions in discrete action spaces, while the SAC algorithm is designed for continuous action spaces. To adapt the SAC algorithm for discrete action spaces, it was modified to iSAC. Additionally, the prioritized experience replay (PER) method [12] was introduced to accelerate the learning process while maintaining stability, improving the efficiency and accuracy of computation offloading and enhancing the resource utilization rate of the CPN for intelligent vehicles.
- (2) A simulation environment has been designed and implemented to encompass the random variations of vehicles and tasks, communication links, and edge servers, achieving unified management of global information variables. This environment can simulate the task transmission process, including the transmission medium, path, and distance, as well as the time that tasks reside on servers, such as queuing and computation time. To validate

the system's reliability, simulations of server energy consumption and loads have also been conducted.

(3) Within the simulation experimental environment of this study, the performance of the iSAC, PER-iSAC, the original SAC, and several other offloading strategies based on common deep reinforcement learning algorithms was tested and compared in terms of task offloading. An in-depth analysis was conducted and insights were provided regarding the results.

2. Related Work

In recent years, numerous studies on resource scheduling have emerged across various fields. These studies have provided some reference methods and inspiration for this research.

In the field of vehicular edge computing (VEC), several recent works have applied DRL to enhance task offloading and resource allocation. Shi et al. [13] proposed a task offloading decision-making algorithm using a deep deterministic policy gradient (DDPG)-based approach in a three-tier "cloud-edge-vehicle" VEC architecture. Their TODM-DDPG algorithm optimized task scheduling and offloading proportions, reducing total system cost—including delay and energy consumption—by 13% compared to DQN and AC. However, the work focused on general computational tasks, not those specifically addressing image segmentation, limiting its applicability in high-precision Internet of Vehicles (IoV) scenarios. Wu et al. [14] introduced a DRL-based scheme for task offloading and load balancing using the twin delayed deep deterministic policy gradient (TD3) algorithm. By integrating the technique for order preference by similarity to the ideal solution (TOPSIS), their method achieved better server selection and reduced average system cost by 7.2% compared to TD3-TR and 61.1% compared to ARO. Nonetheless, their model assumes tasks are only offloaded within the current RSU coverage, failing to address challenges arising from high-mobility vehicles transitioning across RSUs.

To further address latency-sensitive offloading and heterogeneous resource demands, Zhong et al. [15] introduced a resource allocation mechanism with strict latency guarantees for computing power networks (CPNs). Their approach considered heterogeneity in computation and communication resources while ensuring bounded delay, offering insights into latency-aware scheduling strategies applicable to vehicular and edge computing scenarios.

In the domain of image-related edge computing, Nie et al. [16] proposed a DRL-assisted task offloading and resource allocation algorithm for object detection in autonomous vehicles. The DRPL algorithm combined DRL with piecewise linearization to maximize a time utility function, achieving 12.8% to 17.4% improvements. Although closely related, this work focused on object detection rather than image segmentation and did not explore multi-vehicle coordination or network congestion issues. Yuan et al. [17] proposed a hierarchical flow learning framework for low-light image enhancement. Their model improved visibility and structural detail preservation, which is critical for upstream visual tasks such as image segmentation in low-visibility vehicular environments. This technique enhances the quality of pre-processed input data, which can directly affect the accuracy of downstream segmentation tasks. Furthermore, Liu et al. [18] conducted a study on medical image edge computing, in which a DRL-based strategy was developed to offload dependent tasks modeled as directed acyclic graphs (DAGs), balancing delay and energy consumption. While not in the IoV domain, this work offers valuable insights into DRL applications for image-related tasks with dependency structures, which is relevant for collaborative computing.

Addressing integrated sensing and communications, Xue et al. [19] introduced the Vehicle-Assisted Fusion Perception Offloading (VAFPO) scheme using a state-normalized DDPG algorithm (SNDPO). By leveraging ISAC's perception features, the system dynam-

ically chose whether to offload data to auxiliary nodes to minimize delay and energy consumption. However, the work emphasized general sensing data and not high-precision visual tasks like image segmentation.

Security concerns also play a crucial role in distributed edge environments. Yang et al. [20] proposed a fine-grained intrusion protection system that enables inter-edge trust transfer using contextual behavior modeling and federated decision making. Their system significantly improves defense granularity and adaptability, which is essential for protecting data integrity and computation trust during task offloading among edge nodes in vehicular networks.

In other application domains, Hossain et al. [21] proposed a smart grid demand response coordination method based on ultra-reliable low-latency communications and the MuZero algorithm, highlighting the importance of reliability and response time.

To address the reviewer's suggestion, we have incorporated several recent works published between 2023 and 2025 that focus on DRL-based task offloading in edge computing. Table 1 summarizes these contributions by analyzing their system models, algorithmic solutions, evaluation metrics, and limitations. This structured comparison highlights the novelty and advantages of our PER-iSAC framework, especially in handling image segmentation tasks and resource allocation under dynamic conditions.

Table 1. The summary table of the above task-offloading papers.

Reference	System Model	Solution Approach	Performance Metrics	Limitations
Shi et al. [13]	Cloud-edge-vehicle VEC, partial offloading	TODM-DDPG with actor-critic framework	System cost reduction	Not considering task dependencies
Wu et al. [14]	Multi-vehicle, multi-server VEC, MDP	TOLB with TD3 and TOPSIS	System cost reduction	Not considering vehicular mobility
Zhong et al. [15]	CPN with non-independent subtasks	Optimized cycle, dynamic bandwidth	Latency violation probability reduction	Not adequately addressing task correlations, resource preferences, or modeling diverse tasks and heterogeneous resources
Nie et al. [16]	MEC for self-driving, end-edge collaboration	DRPL with DNN, permutation grouping	Time utility improvement	Not considering task dependencies and the priority of image tasks
Liu et al. [18]	RIDM tasks as DAG, edge computing	DCDO-DRL with S2S and SAC	Execution utility improvement	Without considering vehicular mobility and the priority of image tasks
Xue et al. [19]	VEC with ISAC, joint sensing-computation	VAFPO with SNDAO, priority factor	System overhead minimization	It is not yet clear

The aforementioned studies have primarily focused on reducing time and energy consumption; however, they have not taken into account the accuracy rate of tasks and resource allocation. When tasks are allocated to servers with high loads, it can lead to

poor timing and execution efficiency, indicating resource mismatches in task allocation. To address this issue, a task offloading system that integrates the PER method and the iSAC algorithm has been designed in this paper. The experimental results demonstrate that PER-iSAC can achieve better resource utilization rates compared to baseline offloading strategies, ensuring the full utilization of computing resources and a lower error rate.

3. System Architecture

3.1. Computing Power Network

With the widespread deployment of edge servers and RSUs, it has become more convenient and accessible for connected cars to utilize these vast distributed computing resources. Edge computing has witnessed remarkable advancements in recent years, enabling users to access a wide range of applications and services on their mobile devices [22]. However, the computing power of a single vehicle or node is very limited, and for compute-intensive tasks, this can lead to increased computational loads and prolonged task processing time. Moreover, the lack of an effective collaborative mechanism between the edge nodes and cloud computing nodes results in low efficiency and a high task allocation error rate. How to more efficiently utilize these computing resources has become an urgent problem to solve. Consequently, the CPN [23] has emerged, connecting distributed edge and terminal nodes to form a much more powerful network and allocating resources through a unified scheduling algorithm. The research presented in this paper is set within the scenario of a CPN, utilizing scheduling algorithms to enable image segmentation tasks to better acquire computing resources.

The CPN consists of three layers: cloud, edge, and terminal. The terminal devices are at the bottom layer, closest to the users, and are primarily responsible for image sensor data collection and preprocessing. Edge servers are in the middle edge layer, situated between the cloud and the terminal layers, and are responsible for processing the computational tasks uploaded from terminal devices. Cloud servers are in the top layer. They are responsible for model training and distributing models to edge servers, enabling each edge server to handle image segmentation tasks. When necessary, they also process edge computing tasks. Edge servers are close to the vehicles at the terminal layer, resulting in small transmission delays, but their storage capacity and computing power are less than those of the cloud layer. Cloud servers have strong computing capabilities and large storage capacities, enabling them to efficiently handle computing tasks. However, cloud servers are far from the terminal layer, leading to significant data transmission delays [24]. The three-tier architecture of the CPN is shown in Figure 1.

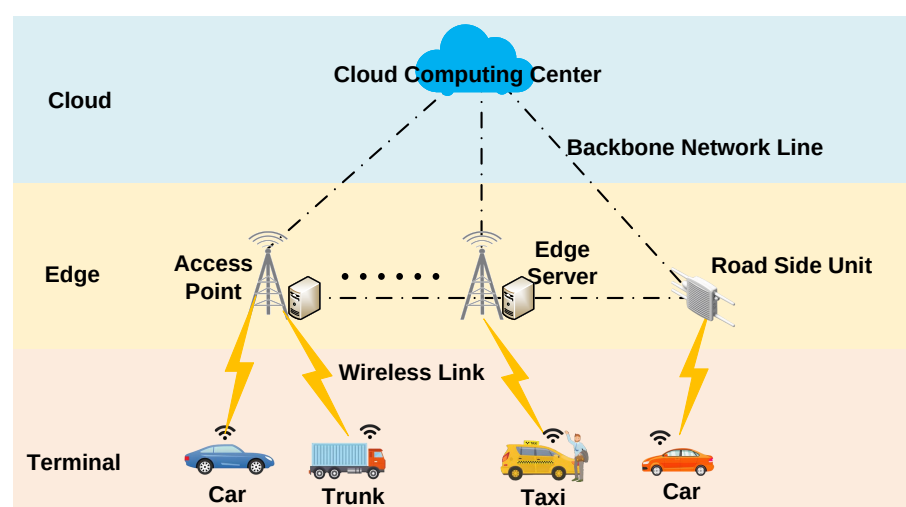


Figure 1. CPN architecture.

3.2. Computing Power Allocation System

Given the high real-time requirements and huge computational demands of image segmentation tasks in smart vehicles, coupled with the insufficiency of terminal computing power, this study has designed a computing power allocation system within the CPN. The system composition is shown in Figure 2. This computing power allocation system is not only highly adaptable and compatible but also stable and reliable. Through this system, it is possible to significantly enhance edge computing efficiency, reduce the energy consumption of connected cars, ensure the timeliness of task completion, and improve the overall capability of edge computing in various IoV computing-power allocation scenarios.

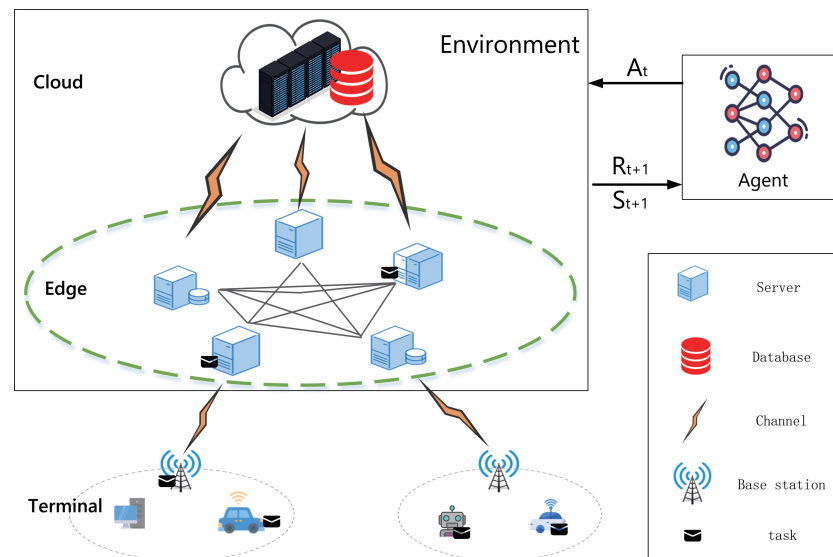


Figure 2. Computing power allocation system.

Each server acts as both a computing node and a computing-power allocation model. The computing power allocation system is deployed on edge servers. As shown in Figure 2, the overall architecture of the system consists of several key components, each designed to enhance the efficiency and rational allocation of computing power. First, the system initializes and manages environmental parameters, which include server performance metrics, communication line performance metrics, and the size of task data, among other specific simulation settings. This is to ensure the rationality of resource allocation, enabling the proposed PER-iSAC to function more effectively in the real world. Second, this paper simulates a real-world cloud–edge network architecture. Its role is to mimic the actual network environment to generate environmental parameter data. Specifically, it simulates the working scenarios of vehicles, tasks, servers, and communication links. In this environment, PER-iSAC can dynamically obtain parameter data, forming a state space. The vehicular and edge servers make decisions based on the state space and action space parameters, and PER-iSAC provides agents with rewards and states for the next time slot based on the actions taken by them. Additionally, the agent is the core of decision making. It is designed to accommodate various algorithms and this paper takes the PER-iSAC algorithm as an example, which can generate the optimal allocation strategy through experience replay and environmental state analysis, thereby reducing time consumption and ensuring that vehicle demands are met, while also enhancing edge computing power. Moreover, PER-iSAC can also perceive the changes in the environment after an agent makes a move. Finally, the agent adjusts the allocation strategy in real time based on the rewards returned by the environment, gradually improving the overall optimization effect of PER-iSAC. In this architecture, cloud servers play a crucial role in model retraining and distribution. When edge servers reach their computational capacity, tasks are offloaded to cloud servers to

ensure continuous processing. This offloading occurs in scenarios where the computational demands exceed the available resources at the edge, such as during peak usage times or when handling particularly large datasets. By leveraging cloud resources, the system can maintain performance and meet the real-time requirements of image segmentation tasks, even under high load conditions.

The allocation of computing power is essentially a data transfer process that involves reasonably assigning tasks to various edge servers and cloud servers based on task requirements. Taking image segmentation tasks as an example, the specific process is as follows. First, the vehicle collects a certain batch of images through its sensing devices (such as cameras and lidars for 3D point clouds) and performs preliminary processing. Since the vehicle has some computing power, it can handle preliminary processing tasks such as image denoising and cropping. The size of the preprocessed image data, selected model, and computing power requirements are then uploaded to the nearest edge server as the task parameter information. Second, after receiving the task parameters, the edge server converts it with the server information into state space data. Third, the agent makes the current optimal edge server selection strategy based on the current state. Fourth, the system returns the optimal strategy to the vehicle and the vehicle sends the batch of image data to the edge server determined by the computing power allocation strategy for image segmentation. Fifth, the edge server executes the image segmentation task and returns the results to the vehicle. If the edge server is at capacity, the task may be redirected to a cloud server for processing, ensuring that the task is completed without further delay. Finally, the system rewards or punishes the PER-iSAC agent based on the reward mechanism.

4. Model Establishment

4.1. Problem Formulation

The actual CPN scenario is as follows. First, each image segmentation task requires computing resources on the edge nodes (GPU and memory resources) and the allocated tasks must not affect the normal operation of other tasks on the same edge node. Second, due to the uniqueness of the vehicle and edge server, e.g., the different geographical locations and communication bandwidths of each vehicle, the time cost and energy expenditures for each task vary. Third, the requirements for each task are diverse. Factors like the task type and image data size lead to varying demands on computing resources. Therefore, the task allocation scheme should ensure that each task receives the optimal computing resources, thereby reducing the completion time and the costs of image segmentation tasks.

This paper views the task allocation problem as a classic resource scheduling optimization problem. Tasks can be considered as image data that need processing, and edge and cloud servers are the computational resources that handle these tasks. The main objective is to find the optimal task allocation strategy so that the distribution of computational resources among tasks across various servers is optimized.

4.2. Computing Power Model

Computing power is defined as the capability required to accomplish certain tasks, including logical/parallel computing, neural network acceleration, etc. The main hardware for computing includes Central Processing Units (CPUs), Graphics Processing Units (GPUs), Tensor Processing Units (TPUs), Neural Processing Units (NPU), etc. CPUs are suitable for logical calculations/operations and general-purpose computing hardware but have poor parallel processing capabilities and struggle with large-scale high-definition image data. In contrast, GPUs have a large number of computing units and a pipeline-style workflow, offering strong parallel processing capabilities and being well-suited for graphic and image processing. The basic unit of measurement for computing power is FLOPS (Computing

Power Units: 1 KFLOPS = 10^3 FLOPS, 1MFLOPS = 10^6 FLOPS, 1GFLOPS = 10^9 FLOPS, 1TFLOPS = 10^{12} FLOPS, 1PFLOPS = 10^{15} FLOPS, 1EFLOPS = 10^{18} FLOPS), which stands for the number of floating-point operations per second. Considering an actual CPN, the most widely used combination of computing hardware is CPU plus GPU, with GPUs excelling in image processing and matrix operations. Therefore, this paper considers GPUs as the hardware for computing power modeling.

4.3. Objective Function

(1) To achieve the rational allocation of computing power in the CPN for the IoV, it is necessary to consider entities such as vehicles, servers, tasks, and network links. Let the set of servers be S , the set of tasks be T , and the set of network links be L . Let Q_s denote the set of tasks in the computation queue at server s . To minimize task completion time, minimize energy consumption, and maximize resource utilization, the comprehensive optimization objective function is defined as in Equation (1):

$$\min \quad \alpha \cdot \sum_{j=1}^m T_{total}^{t_j} + \sum_{i=1}^n (\beta \cdot E^{s_i} - \gamma \cdot U^{s_i}) \quad (1)$$

where α , β , and γ are weights to balance latency, energy consumption, and computational resource utilization. This objective function encapsulates the goals of optimizing the allocation of computing resources in such a way that it balances the efficiency of task completion, the conservation of energy, and the maximization of the use of available resources within the network.

(a) The total latency is the total time from when the task data is sent from the vehicle terminal, processed by the edge server, and the result is returned to the vehicle. It includes transmission delay, queuing delay, and processing delay. The transmission delay includes the task upload delay and the task download delay, where the sizes of data uploaded and downloaded are different and the links they traverse also differ. The transmission delay can be calculated using Equation (2):

$$T_{trans}^{t_j} = \frac{D_{task}^{t_j}}{B_{upload}^{l_k, t_j}} + \frac{D_{result}^{t_j}}{B_{download}^{l_k, t_j}} + \frac{L_{upload}^{l_k, t_j} + L_{download}^{l_k, t_j}}{v}, \quad (2)$$

where D_{task} is the size of the task data and B_{upload} is the bandwidth of the link from the vehicle j to the server i . D_{result} is the size of the task result, and $B_{download}$ is the bandwidth overheads of downloading the task result from the server to the vehicle. L_{upload} represents the link length for task uploading, $L_{download}$ represents the link length for the return of task completion results, and v is the propagation speed of the signal. The processing delay is the time costs for the edge server to provide computing resources based on the task requirements, denoted as $P_{processing}$ (unit: ms, milliseconds), defined as follows. R_{task} is the task's computing resource requirement and C_{server} is the total computing power of the edge server. The processing delay can be calculated using Equation (3):

$$P_{processing}^{t_j} = \frac{R_{task}^{t_j}}{C_{server}^{s_i}} \quad (3)$$

Due to hardware limitations, edge servers have a maximum limit on the number of computing tasks they can handle. Assuming that an edge server receives a task and finds that it has reached the maximum number of tasks, the task will enter a queue, potentially increasing the task's latency. The queuing delay is recorded using system time steps: if a

system time step is set to 0.001 s, then queuing delay W is equivalent to the number of time steps the task spends waiting in the queue.

To ensure the reliability of the computing power scheduling model, this paper considers a CPN scenario where servers are fully loaded when modeling the extended task processing time. The total time is then given by Equation (4):

$$T_{total}^{tj} = T_{trans}^{tj} + P_{processing}^{tj} + W^{tj} \quad (4)$$

(b) Energy consumption is determined by the power of the edge server during task processing and the time spent on processing the task. The power of the edge server varies between the idle and loaded states; the higher the power, the greater the energy consumption and the higher the costs incurred by the server. This paper uses P_{idle} to denote the power consumption in the idle state, P_{full} to denote the power under full loads, and μ^s to denote the resource utilization rate (ranging from 0 to 1). The energy consumption for the computing task t on the edge server s is defined as in Equation (5):

$$E^{tj,si} = (P_{idle}^{si} + (P_{full}^{si} - P_{idle}^{si}) \cdot \mu^{si}) \cdot T_{proc}^{tj} \quad (5)$$

where the current power per unit time of the edge server is given by Equation (6):

$$P^{si} = P_{idle}^{si} + (P_{full}^{si} - P_{idle}^{si}) \cdot \mu^{si} \quad (6)$$

(c) The resource utilization rate is used to measure the current usage of the edge server, including the weighted average of the utilization rates of computing power and memory. The resource utilization rate is defined in Equation (7).

$$U^{si} = w_1 \cdot U_C^{si} + w_2 \cdot U_M^{si} \quad (7)$$

The computing power utilization rate is determined by the total computing power demand of all tasks currently allocated to server s divided by the total computing power of s under full load, which is defined as follows in Equation (8):

$$U_C^{si} = \frac{\sum_{j=1}^h R_{task}^{tj}}{C_{server}^{si}}, \quad t_j \in Q_{si} \quad (8)$$

Similarly, let M_{task} denote the size of memory occupied by the task and let M_{server} denote the total memory capacity of the edge server. The memory utilization rate is then given by Equation (9):

$$U_M^{si} = \frac{\sum_{j=1}^h M_{task}^{tj}}{M_{server}^{si}}, \quad t_j \in Q_{si} \quad (9)$$

Then, the comprehensive resource utilization rate can be obtained by Equation (10):

$$U^{si} = w_1 \cdot \frac{\sum_{j=1}^h R_{task}^{tj}}{C_{server}^{si}} + w_2 \cdot \frac{\sum_{j=1}^h M_{task}^{tj}}{M_{server}^{si}}, \quad t_j \in Q_{si} \quad (10)$$

The weights for each type of resource must respect $w_1 + w_2 = 1$

(2) Constraints are as follows:

In the CPN, let x_{ts} denote task t allocated to edge server s , where S is the set of servers. For task t and server s , it is necessary to ensure that the total computing power and memory requirements of the tasks allocated to the server do not exceed the server's total computing power and memory capacity. These constraints can be expressed as in Equations (11) and (12):

$$\sum_{t=1}^T x_{ts} \cdot R_{task}^{tj} \leq C_{server}^{s_i}, \quad s_i \in S \quad (11)$$

$$\sum_{t=1}^T x_{ts} \cdot M_{task}^{tj} \leq M_{server}^{s_i}, \quad s_i \in S \quad (12)$$

Each task can only be assigned to one server. To ensure that this constraint is met, the scheduling algorithm must be designed to prevent any task from being duplicated across multiple servers. This is crucial for maintaining the integrity of the system and ensuring that resources are allocated efficiently. This approach helps to avoid conflicts and ensures that each server operates at its maximum potential without overloading, which could lead to decreased performance or system failures. This constraint is defined in Equation (13):

$$\sum_{s=1}^m x_{ts} = 1, \quad \forall t \in T \quad (13)$$

The transmission time for each task must not exceed the predetermined maximum latency. This constraint is defined in Equation (14):

$$T_{trans}^{tj} \leq T_{max}^{tj} \quad (14)$$

Adhering to this constraint is essential for ensuring that tasks are completed within the required timeframes, which is particularly important in the IoV where real-time processing is often a critical factor. Exceeding the maximum latency can lead to delays in task execution, which may have cascading effects on downstream processes and ultimately impact the overall performance of the IoV system. Therefore, the scheduling algorithm must efficiently manage the allocation of tasks to edge servers, taking into account the bandwidth limitations and the current network conditions. This optimization not only helps in meeting the latency requirements of the IoV but also enhances the responsiveness of the CPN, which is vital for maintaining the quality of service and user satisfaction in time-sensitive IoV applications. The physical distance between the vehicle and the edge server must not exceed the coverage range of the link, and the distance from the vehicle to the server must be less than the length of the transmission link. This constraint is defined as follows in Equation (15):

$$d < L^{l_k} \quad (15)$$

This constraint ensures that the communications between the vehicle and the edge server is feasible. The distance limitation is crucial for maintaining signal integrity and reducing the potential for data loss or corruption that can occur over long transmission distances. Additionally, this constraint helps ensure that edge servers are strategically placed to service the vehicles within their coverage area and that the overall network remains efficient and scalable. The total completion time for task T must not exceed the maximum completion time allowed for the task, i.e., its deadline, as follows in Equation (16):

$$T_{total}^{tj} \leq T_{max}^{tj} \quad (16)$$

The resource utilization rate of an edge server must be less than 1. This constraint is defined in Equation (17):

$$U^{s_i} \leq 1 \quad (17)$$

This constraint ensures that no edge server is overloaded beyond its capacity, which is crucial for preventing system crashes and maintaining the stability and performance of the CPN.

5. Improved SAC (iSAC) Algorithm

5.1. Algorithm Architecture

In DRL, an agent must learn the optimal policy through interactions with the environment. During this process, the agent faces two main tasks: exploration and exploitation. Exploration refers to the agent's attempt to try different actions to discover new, potentially better strategies, whereas exploitation refers to the agent's selection of actions based on the currently known optimal policy. The SAC algorithm balances these two tasks by maximizing an entropy.

Entropy, a measure of the uncertainty or randomness of a system, is also used in information theory to gauge the uncertainty of information. In reinforcement learning algorithms that maximize entropy, the goal is not only to maximize accumulated rewards but also to maximize the entropy of the policy. This means that the algorithm is encouraged to explore a variety of different actions, even if these actions do not appear to be the optimal choice at the current moment.

The original SAC algorithm achieves the maximization of the entropy by introducing an entropy term in the objective function. Specifically, the objective function of SAC consists of two parts: one is the expectation of accumulated rewards and the other is the expectation of the policy entropy. By adjusting the weights of these two parts, SAC can maximize accumulated rewards while maintaining the randomness of the policy, thus avoiding premature convergence to local optima. Equation (18) represents the overall maximization goal of SAC, where γ is the discount factor and α is the hyperparameter that balances the trade-off between the rewards and the entropy, also known as the temperature coefficient.

$$\max_{\pi} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] + \alpha \mathbb{E}_{(s,a) \sim \pi} [\log \pi(a|s)] \quad (18)$$

In the SAC algorithm, the policy network is responsible for generating the probability distribution of actions, while the value network is responsible for estimating the state-action value function. The agent interacts with the environment according to the current policy, collecting information such as states, actions, rewards, and new states, which are used to update the value network and the policy network. When updating the policy network, SAC considers not only maximizing the expected reward but also maximizing the policy entropy, as shown in Figure 3. This is achieved by computing the policy gradient and updating the weights of the policy network.

An important feature of the SAC algorithm lies in its ability to adaptively adjust the entropy weight. During the training process, entropy weight α is automatically adjusted by another parameter, which allows SAC to automatically balance exploration and exploitation in different tasks and environments without manual interventions. Additionally, SAC is off-policy, which means it can utilize past data through a replay buffer for repeated training. This off-policy updating method offers greater stability compared to online algorithms like A3C.

The improvements made in this study to the SAC algorithm mainly involve the following aspects:

(1) Since the SAC algorithm is a reinforcement learning algorithm suitable for continuous action spaces and the task offloading problem in IoV belongs to discrete space decision making, this paper proposes iSAC, which is applicable to discrete space decisions. As shown in Figure 3, the original actor network only had fully connected layers and ReLU activation functions. This paper adds normalization layers and softmax activation functions. The normalization layer standardizes the output, thereby enhancing the model's representing power. The main reason for using a normalization layer in the output is to prevent the

softmax function's output from being singular. The softmax activation function, as defined in Equation (19), transforms the policy output by SAC into a probability distribution vector, then edge servers can be sampled randomly according to these probabilities.

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^N \exp(z_j)} \quad (19)$$

(2) Based on the characteristics of the CPN and the purpose of computation offloading in the IoV, the state space and reward function are redefined. The state space mainly includes the remaining resources and the load status of edge servers. Since it is necessary to obtain the time tasks spend on edge servers, rewards may be delayed. Therefore, the reward function is divided into immediate decision rewards and deferred rewards.

(3) Introducing PER. The traditional experience replay mechanism uses uniform sampling, thus it may overlook some critical samples and not all samples contribute equally to the update of the value function. The core idea of PER is to prioritize the sampling of experiences with larger Temporal Difference (TD) errors. The TD error is a measure of the deviation between the current value estimate and the target value, defined as in Equation (20):

$$\delta = r + \gamma Q(s', a') - Q(s, a) \quad (20)$$

where r is the immediate reward, γ is the discount factor, $Q(s, a)$ is the current value estimate, and $Q(s', a')$ is the value estimate of the next state. The priority P_i of a sample is related to the TD error, defined as in Equation (21):

$$P_i = |\delta_i| + \epsilon \quad (21)$$

ϵ is a positive number that approaches 0, and samples with higher priority are more likely to be sampled.

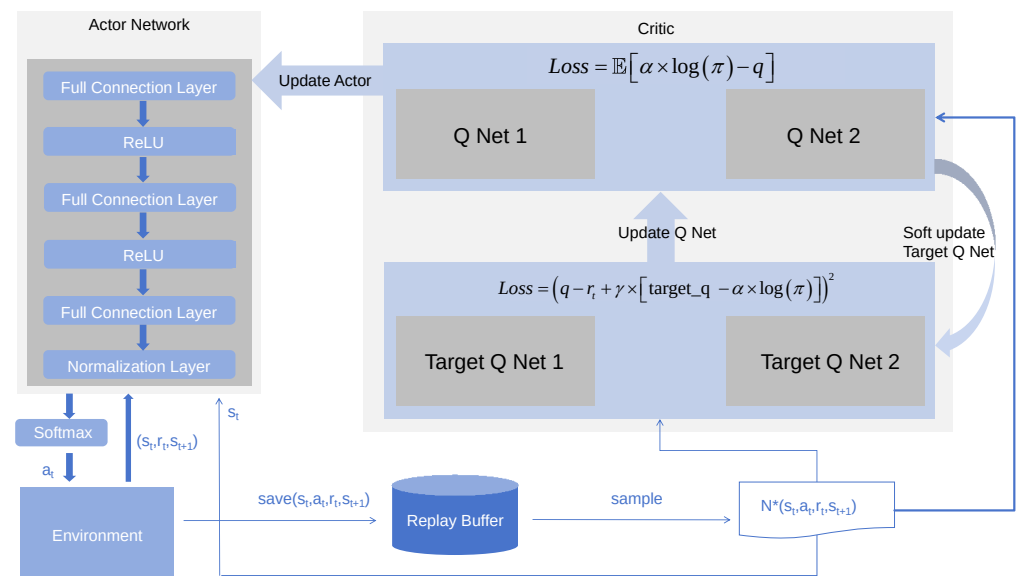


Figure 3. iSAC algorithm architecture.

5.2. MDP Engineering

Since DRL methods possess stronger generalization capabilities and applicability, for the optimization of computing resource allocation problems addressed in this paper, applying Markov Decision Process (MDP) modeling enables the more efficient discovery of optimal solutions.

5.2.1. State Description

In the context of the IoV, a computing power allocation process encompasses the processes of task offloading, task queuing, task computing, and result returning. In the preset scenario, the state space consists of six scores, $\{0, 0, 0, 0, 0, 0\}$, corresponding to the six servers in the action space. Initially, the system needs to collect server load and task information. Then, a score is assigned. If the remaining storage capacity of the server is greater than the storage requirement of the task, add 1 to the corresponding position in the state space set, otherwise do not add. If the server's remaining computing power is greater than the task's computing power requirement, add 1 to the corresponding position in the state space set, otherwise do not add. Finally, the system will calculate the transmission time and computation time overhead (excluding task queuing time and communication delay) of the computing task to each server and select the server with the minimum time cost to add 1 to its score. Such a state space not only expresses the size of the task, the computing power requirements, the server load, and the task time components but also reduces the dimensionality of the state space, thereby accelerating the convergence of the algorithm. Therefore, state space S can be represented as in Equation (22):

$$s(t_j) = \{score_0, score_1, score_2, score_3, score_4, score_5\} \quad (22)$$

5.2.2. Action Description

The agent in iSAC will make a decision to select an edge server based on the current state and task requirements. In the IoV in this paper, a total of six edge servers with different performance parameters are set up, so the action space consists of all the optional edge servers. The action space can be represented as in Equation (23).

$$action(s, t) = \{s_0, s_1, s_2, s_3, s_4, s_5\} \quad (23)$$

5.2.3. Reward Engineering

The core objective of designing the reward function for image segmentation tasks in the IoV is to optimize task allocation to achieve a comprehensive optimum in terms of task completion time and allocation error rate. Integrating the previous optimization objectives and constraints, the following reward function can be designed:

(1) Reward function. iSAC will estimate the total computation and transmission time for image segmentation tasks on each edge server by synthesizing global information and selecting the server with the shortest task completion time as the reference edge server. If the selection matches well, the agent will receive a reward of 1, as shown in Equation (24).

$$R_T = 1 \quad \text{if} \quad s_i = s_m \quad (24)$$

(2) Penalty factor. If the total time T_{total} exceeds the maximum completion time $T_{max_completion}$, it will affect the performance of the CPN for IoV and be deemed as an incorrect allocation by the resource and task allocation system. This is reflected in the penalty term shown in Equation (25):

$$R_T = R_T - 1 \quad \text{if} \quad T^{total} > T^{max} \quad (25)$$

If the edge server is overloaded and unable to accommodate the current task, it will result in an allocation error. The corresponding penalty is given in Equation (26):

$$R_M = -1 \quad (26)$$

Finally, the overall reward function can be obtained, as defined in Equation (27):

$$R = R_T + R_M \quad (27)$$

5.3. Algorithm Implementation

First, the environment, deep neural network, and experience pool are initialized. Hyperparameters are set, and the deep neural network adopts He initialization for parameters. When the experience buffer is empty, the default priority of experiences is set to 1.0. Before the experience pool reaches the batch size, random actions are obtained using the actor network for decision making and states, actions, rewards, and the next states are stored in the experience pool. Once the experience pool reaches the batch size, every task will sample n experiences based on the priority of the samples in the experience pool and then train the samples, updating the policy network and value network. Finally, the temperature coefficient α and the priority of the sampled n experiences are updated. Algorithm 1 presents the pseudocode of PER-iSAC.

Algorithm 1 PER-iSAC

Input: discount factor γ , temperature coefficient α , soft update coefficient τ , batch size n , learning rate

Output: policy

```

1: Initialize Actor and Q Net 1, Q Net 2, Target Q Net 1, Target Q Net 2, hyperparameters
2: Initialize Replay Buffer
3: for each  $episode \in [1, M]$  do
4:   Initialize environment
5:   for each  $t \in [1, T]$  do
6:      $vector = Actor(t)$ 
7:      $p(t) = softmax(vector)$ 
8:      $a(t) = \text{sampling from action space based on } p(t)$ 
9:      $s(t+1), r(t) = \text{environment exec } a(t)$ 
10:    save  $(s(t), a(t), r(t), s(t+1))$  in Replay Buffer
11:    compute the priority  $p$  in Replay Buffer
12:    if  $len(ReplayBuffer) > n$  then
13:      sample  $n$  samples from Replay Buffer according to  $p$ 
14:      train with  $n$  samples
15:      update Q Net 1, Q Net 2, Actor with  $\alpha$ 
16:      update Target Q Net 1, Target Q Net 2
17:      update  $\alpha, p$ 
18:    end if
19:  end for
20: end for

```

6. Experiment Results

6.1. Simulation Settings

Based on the cloud–edge–terminal network structure, a simulation environment is constructed. In this study, all experiments are implemented using the Python language and Gym and PyTorch frameworks. The Python version is 3.8, the PyTorch version is 2.4.0, and the Gym version is 0.26.1. The simulation environment runs on a computer equipped with an Intel Core i9-14900K processor (manufactured by Intel, Santa Clara, CA, USA), 32 GB of memory, and an NVIDIA RTX 4090 graphics card (manufactured by NVIDIA, Santa Clara, CA, USA). The simulation experiments include six servers {S0, S1, S2, S3, S4, S5} and multiple vehicle terminals. Each vehicle terminal has at least one communication link connected to an edge server. Since image segmentation is related to image processing, handling image data with GPUs is much faster than with CPU computations, so the GPU

is the most important hardware component for measuring the algorithms in this paper. The main performance indicators of the edge servers are shown in Table 2.

Table 2. Main performance indicators of the edge servers.

Servers	GPU Computing Power (TFLOPS)	GPU Storage (GB)	Idle Load Power (W)	Full Load Power (W)
S0	200~250	32	300~500	500~1000
S1	140~160	24	150~350	350~500
S2	130~150	24	150~300	300~500
S3	100~120	16	50~150	200~450
S4	110~130	16	50~150	200~500
S5	100~120	8	50~100	150~300

In addition to edge server parameters, this paper also considers task information and communication link parameters. Task information describes the algorithm models used by tasks and the sizes of task data; the communication link parameters include length and bandwidth, mainly for calculating the transmission time of tasks. They are all necessary in an IoV system, as shown in Table 3.

Table 3. Simulation parameters.

Parameters	Values
Computing Power Requirement	200~4000 GFLOPs
Image Data Size	4.8~180 MB
Model Data Size	10~500 MB
Task Result Coefficient ¹	0.1~0.3
Completion Time Coefficient ²	1.2~1.5
Link Speed	5G: 100 Mbps~10 Gbps Optical Fiber Network: 200~400 Gbps
Communication Range	10~500 m
Communication Delay	1~2 ms

¹ The task result is 0.1 to 0.3 times the original task data size. ² The task deadline is 1.2 to 1.5 times the task completion time.

For image segmentation tasks, the computing resource requirements mainly depend on the deep learning models used. The task offloading of smart vehicles has very high requirements for network latency; typically, an image captured by a vehicular camera is directly transmitted to the edge server for computation. With each vehicle equipped with six to eight cameras, a total of six to eight images are generated at the same time. The floating-point operations required (in FLOPs) for processing a single image via deep learning models (such as U-Net, DeepLabV3+) multiplied by the number of images serves as the task's computing power requirement. The computing power required for a task, which is the computational capability needed during the task execution, depends on factors such as the number of input images for the task and the computational load per image. The computing power requirement of a task can be calculated using Equation (28).

$$R_{task}^{t_j} = N^{t_j} \times R_{image}^{t_j} \quad (28)$$

For the parameter settings in the iSAC algorithm, the actor network has two hidden layers with 512 and 256 neurons, respectively, the critic network has two hidden layers with 256 and 128 neurons, respectively, the gradient descent optimizer is the Adam optimizer, the learning rate is set to 0.0001, the target network uses soft updates, the update parameter

τ is set to 0.005, the size of the experience replay buffer is 10,000, the initial entropy α is 1.5, and the reward discount factor γ is 0.99.

In our simulation environment, the task processing system is implemented as a discrete event simulation (DES) model. Each edge server maintains its own task queue and processes tasks using the First-Come-First-Served (FCFS) rule. While we do not strictly assume Poisson arrivals or exponential service times, the queuing dynamics of each server share similarities with the M/M/1 queuing model. Therefore, our system can be viewed as a parallel set of DES-modeled single-server queues, reflecting the task offloading and scheduling process in edge computing for vehicular networks.

6.2. Experimental Results

To simulate real-world IoV scenarios, the simulation data used in the experiments are randomly generated within a specified range, and multiple simulations are conducted to avoid random errors. To ensure statistical reliability, each experiment was repeated five times with different random seeds. The reported results represent the average performance across all runs. The PER-iSAC policy was trained until the convergence of cumulative rewards was observed. Across all runs, the performance trends were consistent, confirming the stability and robustness of the proposed algorithm.

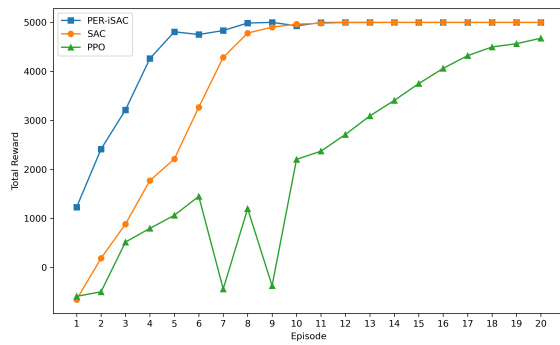
6.2.1. Model Training and Comparative Experiments

In the simulation environment, each experiment will randomly generate a total of 100,000 tasks, with a task arrival rate of 10,000 tasks per second. The simulation tasks used in this study are not derived from a public dataset but are synthetically generated. Key task parameters—such as image data size, model size, and computing power requirements—are generated using normal (Gaussian) distributions centered within their respective value ranges. This method ensures that most tasks fall within a typical operational range while still allowing some variation. The synthetic task dataset provides a controlled and flexible environment for evaluating task offloading strategies in IoV systems.

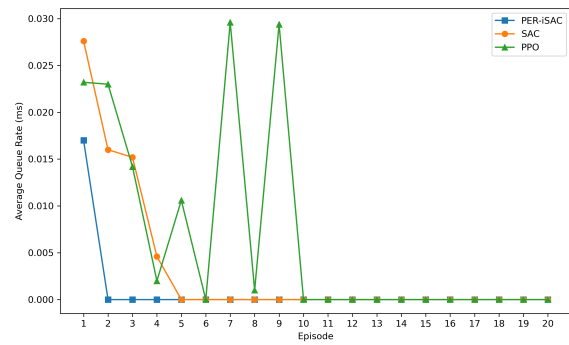
The results are compared and analyzed using the following three scheduling strategies:

- PER-iSAC (Proposed). The scheduling strategy using the PER-iSAC algorithm.
- Standard SAC. The scheduling strategy using the SAC algorithm.
- PPO Baseline. The scheduling strategy using the PPO algorithm [25].

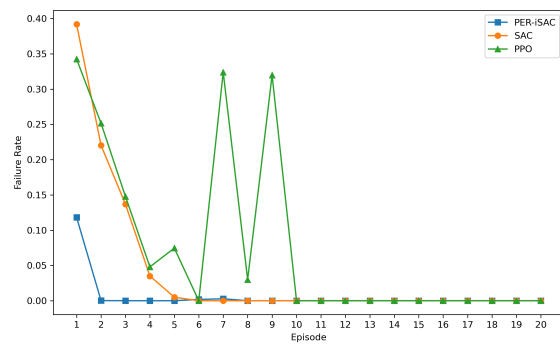
According to Figure 4, through the reward value curve in Figure 4a, it can be observed that PER-iSAC's total rewards rise rapidly in the initial phase (just as quickly as those of Standard SAC) and then level off. This indicates that both methods quickly improve performance during the early learning stage. Moreover, both algorithms converge faster, with smaller and more stable fluctuations. PPO's total rewards increase slowly at first and with greater fluctuations but begin to grow steadily after the 10th episode, eventually reaching a relatively stable state. According to Figure 4b–d, the average queue rate, error rate, and completion time for all algorithms decrease rapidly in the initial phase. This suggests that the algorithms quickly refine their allocation strategy through the system's reward mechanism during the early learning stage. Overall, PER-iSAC is faster than Standard SAC and PPO Baseline in terms of reducing time and decreasing error rates. The main reason for this phenomenon is the introduction of the PER skill in PER-iSAC, which enhances iSAC's utilization of experiences that are of particularly high value but have a small sample size, thereby enabling PER-iSAC to explore actions that yield greater value earlier.



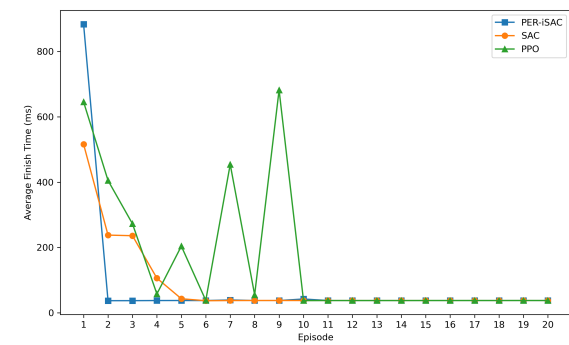
(a) Convergence of Rewards for DRL Methods



(b) Average Queuing Rates



(c) Allocation Error Rates



(d) Average Completion Time

Figure 4. Training convergence plots.

According to Figure 5, all methods select servers in descending order of performance, with higher-performing servers being chosen more frequently, which aligns with the server performance settings in the predefined scenario. However, PPO Baseline has a significantly higher number of selections on Server 0 compared to the other algorithms, likely due to its better performance on that server and the acquisition of more rewards. This also implies that the PPO Baseline tends to select specific servers and does not distribute its selections as evenly across all servers as the other algorithms do. This could be due to PPO Baseline's preference for certain servers during the learning process or its insufficient learning of how to effectively utilize all servers. The reason for PPO Baseline's fluctuation could be that it has not achieved the optimal balance between exploring new strategies and exploiting known strategies. This could lead to it not fully utilizing all servers or not making the most optimal scheduling decisions in certain situations, resulting in instability during the training process.

6.2.2. Performance Metrics of the PER-iSAC Model

In the experiment comparing the average completion times of the algorithms as task size increases, six task sizes were designed: 20 MB, 50 MB, 80 MB, 120 MB, 150 MB, and 180 MB. Each algorithm performed 10,000 task offloads for each task size. Figure 6 presents a comparison of the average task completion times for the three methods under varying task sizes. As the task size increases, the average completion time of the three algorithms all increases. When the task size is small (20 MB to 80 MB), the average completion time of the algorithms is relatively low and grows slowly. When the task size reaches 80 MB, the average completion times of PPO Baseline and Standard SAC begin to increase significantly, while the growth

rate of PER-iSAC is relatively small. When the task size reaches 150–180 MB, the average completion time of PPO Baseline increases the fastest, approaching 500 ms, while the average completion times of PER-iSAC and Standard SAC are about 80 ms and 180 ms, respectively. Under smaller task sizes, there is little difference in the performance of the three algorithms. As the task size increases, the performance of PPO Baseline decreases the fastest and the increase in completion time is the most significant. PER-iSAC shows the best performance across all task sizes, with the lowest average completion time.

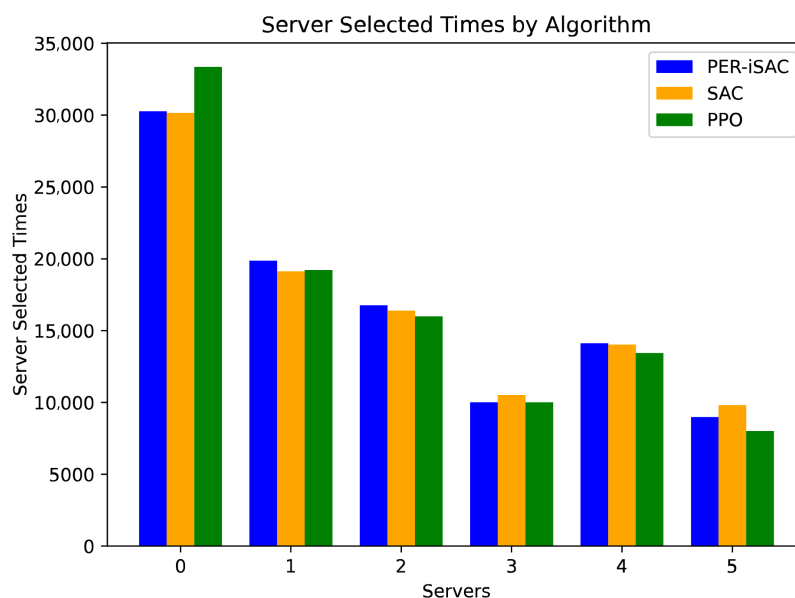


Figure 5. Server selection frequency statistics.

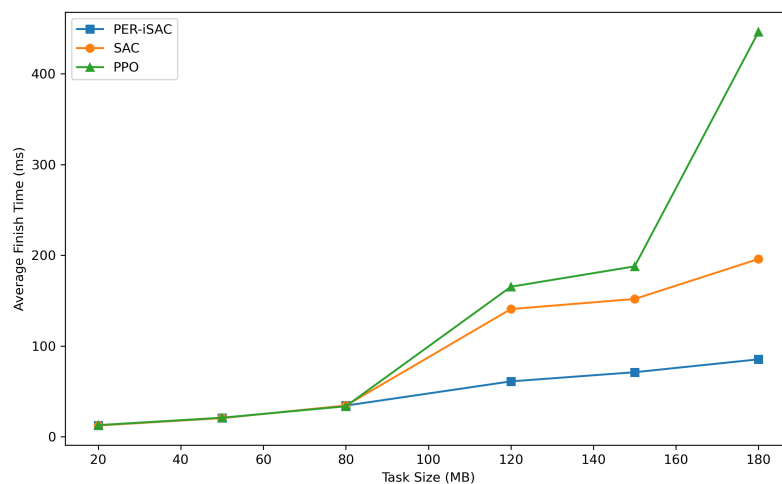


Figure 6. Average completion time for different task sizes.

In order to evaluate the upper-bound performance of the proposed model, in this experiment, the task arrival rate was increased to 100,000 tasks per second, which is ten times higher than the training phase setting. Under the high-intensity configuration, the system processed a total of 5000 tasks. As shown in Figure 7a,b, at the beginning of the experiment, as tasks were offloaded to the server, the server's computing power and storage resource utilization rates gradually increased. When 3000 tasks were offloaded, the storage resource utilization rates of the server approached 100%. When 4000 tasks were offloaded, as shown in Figure 7c, the offloading error rate began to occur. The reason for the error was insufficient storage space on the server. Due to memory capacity limitations, once the memory was saturated, no more tasks could be allocated to the server, so the

average computing power utilization rate remained below 60%. When all 5000 tasks were offloaded, the error rate was approximately 0.8%. Figure 7d presents the energy consumption of edge servers during the same test. The energy usage curve grows steadily and reflects the linear relationship between server load and power consumption, as defined in Equations (5) and (6). These results demonstrate that the proposed algorithm maintains high system efficiency in terms of resource utilization, error rate, and energy consumption control under high-volume task inflow.

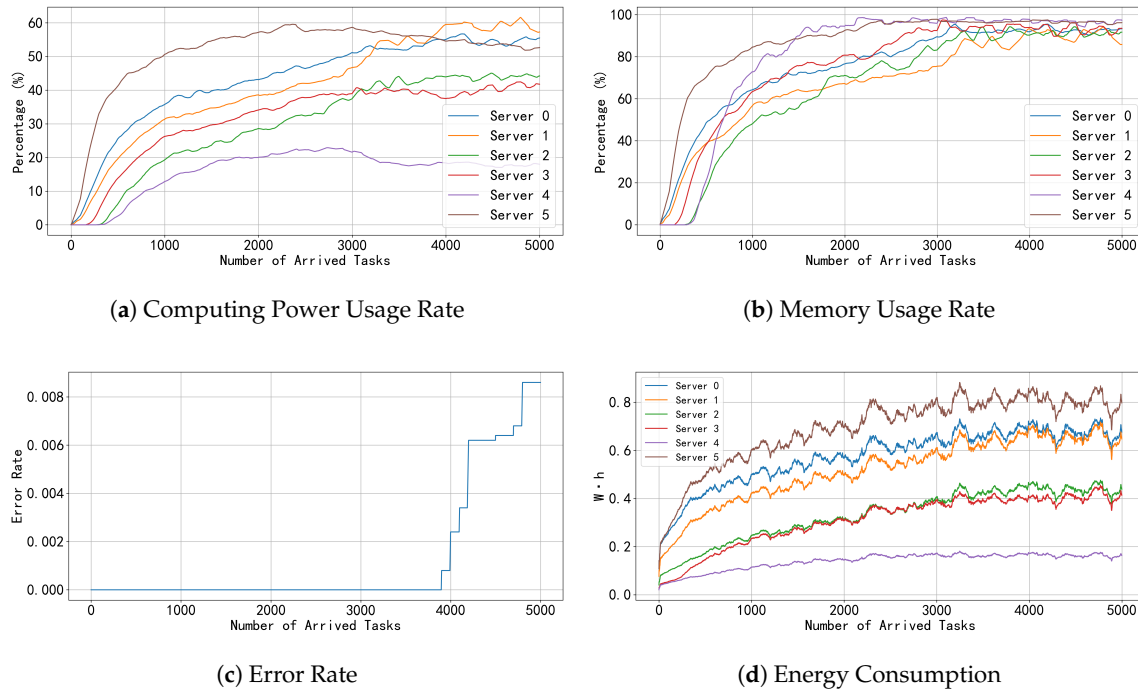


Figure 7. Performance metrics.

Overall, the PER-iSAC model exhibits robust task-handling capability under high load conditions, maintaining system stability, a bounded failure rate, and energy-efficient operation.

7. Limitations and Conclusions

7.1. Limitations

The proposed PER-iSAC algorithm demonstrates promising performance for image segmentation in the IoV, but several limitations remain. First, while the framework is scalable in principle due to its deep reinforcement learning foundation and use of prioritized experience replay, system responsiveness may degrade in large-scale scenarios with high vehicle density and task heterogeneity. Future work will explore distributed or federated learning strategies to enhance scalability. Second, although the iSAC framework is structurally general-purpose and applicable to other vision-based tasks (e.g., object detection, pedestrian tracking), its performance on non-image or multi-modal workloads has not been validated. Third, the current model does not support task prioritization or preemption, limiting its applicability in mixed-criticality environments. Fourth, it does not explicitly model cross-domain edge handovers caused by vehicular mobility, which could impact task offloading performance in dynamic real-world settings. Fifth, cost considerations such as CAPEX and OPEX are not evaluated; while edge offloading reduces per-vehicle hardware costs, future work should include quantitative cost–performance analysis for large-scale deployment feasibility. Lastly, although our evaluation compares PER-iSAC with PPO and Standard SAC, it does not include more recent or advanced approaches such as hybrid offloading methods, federated learning, multi-agent DRL, or LLM-based task

schedulers. We recognize this as a limitation and plan to incorporate such baselines in future studies to further validate and benchmark our method.

7.2. Conclusions

This paper primarily investigates the offloading and allocation of image segmentation tasks for connected cars in the IoV supported by CPNs. The experiments take into account the task transmission delays under varying link bandwidths and transmission distances in 5G and optical networks, as well as the computational delays and queuing delays of servers. Additionally, the data size and model size of image segmentation tasks on smart vehicles, along with their computing power requirements, are considered. This paper designs a PER-iSAC algorithm to implement server selection, optimizing task completion time while maintaining a low decision-making error rate. The simulation experiments confirm that PER-iSAC is effective and more efficient than the PPO and Standard SAC algorithms.

Concluding the analysis, this paper arrived at a fundamental insight: the true value of edge servers within the IoV context extends far beyond mere computational acceleration through task offloading. Their pivotal role lies in enabling the infrastructure for cooperative vehicle behavior, actualized via V2X (Vehicle-to-Everything) communications. These servers act as local coordination and computation hubs, providing the ultra-low latency essential for exchanging safety-critical information between vehicles (V2V), infrastructure (V2I), pedestrians (V2P), and the network (V2N).

This facilitates a paradigm shift from a model where each vehicle relies solely on its onboard sensors and processing power to one of collective intelligence. Data aggregation at the edge servers fuses information (video streams, LiDAR/radar data, GPS coordinates, vehicle status) from numerous road users and infrastructure elements. Based on this enriched, comprehensive view of the traffic environment (akin to a “digital twin” of the local road segment), movement coordination becomes feasible: synchronizing speeds for platooning, optimizing intersection passage, warning about beyond-line-of-sight hazards, and enabling collaborative maneuver planning.

Consequently, an integrated transport network (or grid) is formed, where decisions are made not just at the individual vehicle level, but also at a system level to optimize overall performance. This leads to significant enhancements in safety (reducing collision probability through extended awareness) and traffic efficiency (mitigating congestion, optimizing routes, reducing fuel/energy consumption). Within this complex ecosystem, video information processing, whether performed locally or at the edge, serves as a crucial but not solitary component—it is one data stream feeding the larger process of cooperative transport system management orchestrated by the edge infrastructure.

Author Contributions: Conceptualization, B.Y., W.Z. and H.Y.; methodology, W.Z.; software, W.Z. and A.R.; validation, W.Z., B.Y. and W.L.; formal analysis, W.Z.; investigation, B.Y. and W.Z.; resources, B.Y.; data curation, W.Z., A.R. and W.L.; writing—original draft preparation, W.Z. and B.Y.; writing—review and editing, W.Z. and B.Y.; funding acquisition, H.Y., B.Y. and W.L. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Science and Technology Research Program of Chongqing Municipal Education Commission (Grant No. KJ202301165), the Innovation and Development Joint Fund of Natural Science Foundation of Chongqing (CSTB2024NSCQ-LMX0010), the Scientific Research Foundation of Chongqing University of Technology (Grant No. 0121230236), and the Higher Education Research Project of Chongqing University of Technology (Grant No. 2024YB09). The APC was funded by the Innovation and Development Joint Fund of Natural Science Foundation of Chongqing (CSTB2024NSCQ-LMX0010).

Data Availability Statement: Data are contained within the article; further inquiries can be directed to the corresponding author.

Conflicts of Interest: Author Haitao Yu was employed by the company China Satellite Network Exploration Co., Ltd. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AI	Artificial Intelligence
SAC	Soft Actor–Critic
PER	Prioritized Experience Replay
A3C	Asynchronous Advantage Actor–Critic
PPO	Proximal Policy Optimization
TD	Temporal Difference
CPN	Computing Power Network
IoV	Internet of Vehicles
DRL	Deep Reinforcement Learning
FCFS	First-Come-First-Served
RSU	Road Side Unit

References

- Merzougui, S.E.; Limani, X.; Gavrielides, A.; Palazzi, C.E.; Marquez-Barja, J. Leveraging 5G Technology to Investigate Energy Consumption and CPU Load at the Edge in Vehicular Networks. *World Electr. Veh. J.* **2024**, *15*, 171. [\[CrossRef\]](#)
- Salmane, D.; Mohamed, A.; Khalid, Z.; Driss, B.; Driss, B. Edge Computing Technology Enablers: A Systematic Lecture Study. *IEEE Access* **2022**, *10*, 69264–69302.
- Zhou, S.; Jadoon, W.; Khan, I.A. Computing Offloading Strategy in Mobile Edge Computing Environment: A Comparison between Adopted Frameworks, Challenges, and Future Directions. *Electronics* **2023**, *12*, 2452. [\[CrossRef\]](#)
- Guerna, A.; Bitam, S.; Calafate, C.T. Roadside Unit Deployment in Internet of Vehicles Systems: A Survey. *Sensors* **2022**, *22*, 3190. [\[CrossRef\]](#)
- Mishra, P.; Singh, G. Internet of Vehicles for Sustainable Smart Cities: Opportunities, Issues, and Challenges. *Smart Cities* **2025**, *8*, 93. [\[CrossRef\]](#)
- Lu, S.; Yao, Y.; Shi, W. CLONE: Collaborative Learning on the Edges. *IEEE Internet Things J.* **2021**, *8*, 10222–10236. [\[CrossRef\]](#)
- Dai, Z.; Guan, Z.; Chen, Q.; Xu, Y.; Sun, F. Enhanced Object Detection in Autonomous Vehicles through LiDAR—Camera Sensor Fusion. *World Electr. Veh. J.* **2024**, *15*, 297. [\[CrossRef\]](#)
- Ministry of Industry and Information Technology. *Vehicle Network (Intelligent Connected Vehicles) Industry Development Action Plan*; Ministry of Industry and Information Technology: Beijing, China, 2018.
- Lu, S.; Shi, W. Vehicle Computing: Vision and Challenges. *J. Inf. Intell.* **2022**, *1*, 23–35. [\[CrossRef\]](#)
- Cui, H.; Lei, J. An Algorithmic Study of Transformer-Based Road Scene Segmentation in Autonomous Driving. *World Electr. Veh. J.* **2024**, *15*, 516. [\[CrossRef\]](#)
- Haarnoja, T.; Zhou, A.; Abbeel, P.; Levine, S. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *arXiv* **2018**, arXiv:1801.01290.
- Schaul, T.; Quan, J.; Antonoglou, I.; Silver, D. Prioritized Experience Replay. *arXiv* **2015**, arXiv:1511.05952.
- Shi, W.; Chen, L.; Zhu, X. Task Offloading Decision-Making Algorithm for Vehicular Edge Computing: A Deep-Reinforcement-Learning-Based Approach. *Sensors* **2023**, *23*, 7595. [\[CrossRef\]](#)
- Wu, Z.; Jia, Z.; Pang, X.; Zhao, S. Deep Reinforcement Learning-Based Task Offloading and Load Balancing for Vehicular Edge Computing. *Electronics* **2024**, *13*, 1511. [\[CrossRef\]](#)
- Zhong, A.; Wu, D.; Yang, B.; Wang, R. Heterogeneous resource allocation with latency guarantee for computing power network. *Digit. Commun. Netw.* **2025**, 2352–8648. [\[CrossRef\]](#)
- Nie, L.; Wang, H.; Feng, G.; Sun, J.; Lv, H.; Cui, H. A deep reinforcement learning assisted task offloading and resource allocation approach towards self-driving object detection. *Cloud Comp.* **2023**, *12*, 131. [\[CrossRef\]](#)
- Yuan, X.; Wang, Y.; Li, Y.; Kang, H.; Chen, Y.; Yang, B. Hierarchical flow learning for low-light image enhancement. *Digit. Commun. Netw.* **2024**, 2352–8648. [\[CrossRef\]](#)
- Liu, Q.; Tian, Z.; Wang, N.; Lin, Y. DRL-based dependent task offloading with delay-energy tradeoff in medical image edge computing. *Complex Intell. Syst.* **2024**, *10*, 3283–3304. [\[CrossRef\]](#)
- Xue, J.; Yu, Q.; Wang, L.; Fan, C. Vehicle task offloading strategy based on DRL in communication and sensing scenarios. *Ad Hoc Netw.* **2024**, *159*, 103497. [\[CrossRef\]](#)

20. Yang, B.; Wu, D.; Wang, R.; Yang, Z.; Yang, Y. A fine-grained intrusion protection system for inter-edge trust transfer. *Digit. Commun. Netw.* **2024**, *10*, 2352–8648. [\[CrossRef\]](#)
21. Hossain, M.B.; Pokhrel, S.R.; Choi, J. Orchestrating Smart Grid Demand Response Operations with URLLC and MuZero Learning. *IEEE Internet Things J.* **2024**, *11*, 6692–6704. [\[CrossRef\]](#)
22. Nam, D. H. A Comparative Study of Mobile Cloud Computing, Mobile Edge Computing, and Mobile Edge Cloud Computing. In Proceedings of the 2023 Congress in Computer Science, Computer Engineering, & Applied Computing (CSCE), Las Vegas, NV, USA, 24–27 July 2023. [\[CrossRef\]](#)
23. Tang, X.; Cao, C.; Wang, Y.; Zhang, S.; Liu, Y.; Li, M.; He, T. Computing power network: The architecture of convergence of computing and networking towards 6G requirement. *China Commun.* **2021**, *18*, 175–185. [\[CrossRef\]](#)
24. Andriulo, F.C.; Fiore, M.; Mongiello, M.; Traversa, E.; Zizzo, V. Edge Computing and Cloud Computing for Internet of Things: A Review. *Informatics* **2024**, *11*, 71. [\[CrossRef\]](#)
25. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal Policy Optimization Algorithms. *arXiv* **2017**, arXiv:1707.06347.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.